

lab5_part1

```
1  function lab5_part1()
2  % =====
3  % Lab 5 - Part A / Theme A
4  % Discrete Event Simulation of a Web Server
5  %
6  % File name:
7  %   lab5_part1.m
8  %
9  % Run in Octave with:
10 %   lab5_part1
11 %
12 % Implements:
13 %   A.1 Single DES run for lambda = 100 req/sec, plots and metrics.
14 %   A.2 Queue/system size plots for lambda in {50,100,150,200,250}.
15 %   A.3 95% confidence intervals from n >= 30 independent replications,
16 %       before and after transient removal.
17 %   A.4 Little's Law verification, with and without transient removal.
18 %
19 % Units:
20 %   - lambda is in requests/sec.
21 %   - service times and simulation time are in milliseconds.
22 %   - inter-arrival mean time is 1000/lambda ms.
23 %
24 % Model:
25 %   - Single FIFO server.
26 %   - Each request first becomes a type-1 job.
27 %   - After type-1 completion, the same request becomes a type-2 job and
28 %     re-enters the back of the same FIFO queue.
29 %   - After type-2 completion, the request leaves the system.
30 % =====
31
32 clc;
33 close all;
34
35 % Try to load the statistics package if available.
36 % The code does not require exprnd/lognrnd because local samplers are used.
37 % tinv/norminv are used if available; otherwise fallback approximations are used.
38 try
39     pkg load statistics;
40 catch
41     printf('Note: statistics package was not loaded. Local samplers/fallback quantiles
42 will be used where needed.\n');
43 end_try_catch
44
45 % Fresh RNG state at the start of the script.
46 rand('state', sum(100 * clock));
47 randn('state', sum(100 * clock) + 12345);
48
49 % =====
50 % User-adjustable simulation parameters
```

```

50 % =====
51
52 max_req_A1      = 1e4;
53 max_req_A2      = 1e4;
54 n_runs_A3       = 30;
55 n_req_rep_A3    = 1e4;
56 warmup_fraction_A3 = 0.30; % MQS-guided conservative transient removal
57 warmup_fraction_A4 = 0.30; % same rule for Little's Law comparison
58
59 % =====
60 % A.1 - Single run for lambda = 100 requests/sec
61 % =====
62
63 printf('\n=====\\n');
64 printf('A.1 - Single DES run for lambda = 100 req/sec\\n');
65 printf('=====\\n');
66
67 lambda_A1 = 100;
68 r = sim_webserver(lambda_A1, max_req_A1);
69
70 R1_A1 = mean(r.resp_t1);
71 R2_A1 = mean(r.resp_t2);
72 R_A1  = mean(r.total_resp);
73
74 tp1_A1 = length(r.resp_t1) / (r.t_end / 1000);
75 tp2_A1 = length(r.resp_t2) / (r.t_end / 1000);
76
77 N1_A1 = tw_mean(r.log_q1, r.log_t);
78 N2_A1 = tw_mean(r.log_q2, r.log_t);
79 N_A1  = tw_mean(r.log_q1 + r.log_q2, r.log_t);
80
81 printf('Simulated %d fully served requests up to t = %.4f ms = %.4f sec\\n', ...
82       r.n_served, r.t_end, r.t_end / 1000);
83 printf('Total arrivals observed:      %d\\n', r.n_arrived);
84 printf('R1, type-1 phase response:    %.6f ms\\n', R1_A1);
85 printf('R2, type-2 phase response:    %.6f ms\\n', R2_A1);
86 printf('R end-to-end:                  %.6f ms\\n', R_A1);
87 printf('Type-1 jobs served per second: %.6f\\n', tp1_A1);
88 printf('Type-2 jobs served per second: %.6f\\n', tp2_A1);
89 printf('Mean #type-1 in system, tw:     %.6f\\n', N1_A1);
90 printf('Mean #type-2 in system, tw:     %.6f\\n', N2_A1);
91 printf('Mean total #jobs in system, tw: %.6f\\n', N_A1);
92
93 % A.1 Plot 1: cumulative arrivals and fully served requests.
94 figure(1);
95 clf;
96 plot(r.log_t / 1000, r.log_arrived, 'LineWidth', 1.2);
97 hold on;
98 plot(r.log_t / 1000, r.log_served, 'LineWidth', 1.2);
99 grid on;
100 xlabel('Time (sec)');
101 ylabel('Cumulative requests');
102 title('A.1 - Arrived and fully served requests, lambda = 100 req/sec');
103 legend('Arrived requests', 'Fully served requests', 'Location', 'northwest');

```

```

104
105 % A.1 Plot 2: number of type-1 and type-2 jobs in the system.
106 figure(2);
107 clf;
108
109 subplot(2, 1, 1);
110 plot(r.log_t / 1000, r.log_q1, 'LineWidth', 1.0);
111 grid on;
112 xlabel('Time (sec)');
113 ylabel('Type-1 jobs');
114 title('A.1 - Type-1 jobs in system, queue + service');
115
116 subplot(2, 1, 2);
117 plot(r.log_t / 1000, r.log_q2, 'LineWidth', 1.0);
118 grid on;
119 xlabel('Time (sec)');
120 ylabel('Type-2 jobs');
121 title('A.1 - Type-2 jobs in system, queue + service');
122
123 % A.1 Plot 3: response times and throughput as bar plots.
124 figure(3);
125 clf;
126
127 subplot(1, 2, 1);
128 bar([R1_A1, R2_A1]);
129 grid on;
130 set(gca, 'XTick', 1:2, 'XTickLabel', {'Type 1', 'Type 2'});
131 ylabel('Mean response time (ms)');
132 title('A.1 - Mean response time per phase');
133
134 subplot(1, 2, 2);
135 bar([tp1_A1, tp2_A1]);
136 grid on;
137 set(gca, 'XTick', 1:2, 'XTickLabel', {'Type 1', 'Type 2'});
138 ylabel('Completed jobs per second');
139 title('A.1 - Mean served jobs per second');
140
141 % =====
142 % A.2 - Stationarity across lambda values
143 % =====
144
145 printf('\n===== \n');
146 printf('A.2 - Stationarity across lambda values \n');
147 printf('===== \n');
148
149 S1_mean = exp(1 + 0.5); % E[S1] = exp(mu + sigma^2/2), mu = sigma = 1
150 S2_mean = 0.75; % E[S2] = (0.5 + 1) / 2
151 S_total = S1_mean + S2_mean;
152
153 lambda_stability_bound = 1000 / S_total;
154
155 printf('E[S1] = exp(1.5) = %.6f ms \n', S1_mean);
156 printf('E[S2] = %.6f ms \n', S2_mean);
157 printf('E[S1] + E[S2] = %.6f ms \n', S_total);

```

```

158 printf('Analytical stability condition: rho = lambda*(E[S1]+E[S2])/1000 < 1\n');
159 printf('Stability bound: lambda < %.6f req/sec\n', lambda_stability_bound);
160
161 lambdas_A2 = [50, 100, 150, 200, 250];
162 runs_A2 = cell(1, length(lambdas_A2));
163
164 figure(4);
165 clf;
166
167 for k = 1:length(lambdas_A2)
168     lam = lambdas_A2(k);
169     rho = lam * S_total / 1000;
170
171     if rho < 1
172         status = 'stable';
173     else
174         status = 'unstable';
175     endif
176
177     printf('Running lambda = %d req/sec, rho = %.6f -> %s ... ', lam, rho, status);
178     fflush(stdout);
179
180     runs_A2{k} = sim_webserver(lam, max_req_A2);
181
182     printf('done, t_end = %.4f sec, arrivals = %d, served = %d\n', ...
183         runs_A2{k}.t_end / 1000, runs_A2{k}.n_arrived, runs_A2{k}.n_served);
184
185     q_total = runs_A2{k}.log_q1 + runs_A2{k}.log_q2;
186
187     subplot(length(lambdas_A2), 1, k);
188     plot(runs_A2{k}.log_t / 1000, q_total, 'LineWidth', 1.0);
189     grid on;
190     xlabel('Time (sec)');
191     ylabel('Total jobs');
192     title(sprintf('A.2 - lambda = %d req/sec, rho = %.4f, %s', lam, rho, status));
193 endfor
194
195 printf('\nAnalytical classification for A.2:\n');
196 for k = 1:length(lambdas_A2)
197     lam = lambdas_A2(k);
198     rho = lam * S_total / 1000;
199
200     if rho < 1
201         printf(' lambda = %3d req/sec: rho = %.6f < 1 -> stationary\n', lam, rho);
202     else
203         printf(' lambda = %3d req/sec: rho = %.6f >= 1 -> non-stationary\n', lam,
204             rho);
205     endif
206 endfor
207
208 % =====
209 % A.3 - Confidence intervals before and after transient removal
210 % =====

```

```

211 printf('\n=====');
212 printf('A.3 - Confidence intervals from independent replications\n');
213 printf('=====');
214
215 lambdas_A3 = [100, 180];
216
217 CI1_mean_all = zeros(length(lambdas_A3), 2);
218 CI1_med_all = zeros(length(lambdas_A3), 2);
219 CI2_mean_all = zeros(length(lambdas_A3), 2);
220 CI2_med_all = zeros(length(lambdas_A3), 2);
221
222 CI1_mean_tr = zeros(length(lambdas_A3), 2);
223 CI1_med_tr = zeros(length(lambdas_A3), 2);
224 CI2_mean_tr = zeros(length(lambdas_A3), 2);
225 CI2_med_tr = zeros(length(lambdas_A3), 2);
226
227 % Pilot MQS plots for cutoff visualization.
228 figure(5);
229 clf;
230
231 for li = 1:length(lambdas_A3)
232     lam = lambdas_A3(li);
233
234     pilot = sim_webserver(lam, n_req_rep_A3);
235     q_pilot_total = pilot.log_q1 + pilot.log_q2;
236     mqs_pilot = running_tw_mean(q_pilot_total, pilot.log_t);
237     cutoff_pilot = warmup_fraction_A3 * pilot.t_end;
238
239     subplot(length(lambdas_A3), 1, li);
240     plot(pilot.log_t / 1000, mqs_pilot, 'LineWidth', 1.0);
241     hold on;
242     yl = ylim();
243     plot([cutoff_pilot / 1000, cutoff_pilot / 1000], yl, '--', 'LineWidth', 1.0);
244     grid on;
245     xlabel('Time (sec)');
246     ylabel('MQS(t)');
247     title(sprintf('A.3 - MQS(t), lambda = %d req/sec, cutoff = %.0f%% of run time',
248 ...
249 ... lam, 100 * warmup_fraction_A3));
249     legend('MQS(t)', 'Transient cutoff', 'Location', 'southeast');
250 endfor
251
252 for li = 1:length(lambdas_A3)
253     lam = lambdas_A3(li);
254
255     N1_all = zeros(n_runs_A3, 1);
256     N2_all = zeros(n_runs_A3, 1);
257     N1_tr = zeros(n_runs_A3, 1);
258     N2_tr = zeros(n_runs_A3, 1);
259
260     printf('\nlambda = %d req/sec: running %d independent replications ', ...
261 ... lam, n_runs_A3);
262
263     for rep = 1:n_runs_A3

```

```

264         rk = sim_webserver(lam, n_req_rep_A3);
265
266         N1_all(rep) = tw_mean(rk.log_q1, rk.log_t);
267         N2_all(rep) = tw_mean(rk.log_q2, rk.log_t);
268
269         cutoff_ms = warmup_fraction_A3 * rk.t_end;
270
271         N1_tr(rep) = tw_mean_interval(rk.log_q1, rk.log_t, cutoff_ms, rk.t_end);
272         N2_tr(rep) = tw_mean_interval(rk.log_q2, rk.log_t, cutoff_ms, rk.t_end);
273
274         printf('.');
275         fflush(stdout);
276     endfor
277
278     printf(' done\n');
279
280     [ci1_mean_all, ci1_med_all] = calc_ci(N1_all, 0.05);
281     [ci2_mean_all, ci2_med_all] = calc_ci(N2_all, 0.05);
282
283     [ci1_mean_tr, ci1_med_tr] = calc_ci(N1_tr, 0.05);
284     [ci2_mean_tr, ci2_med_tr] = calc_ci(N2_tr, 0.05);
285
286     CI1_mean_all(li, :) = ci1_mean_all;
287     CI1_med_all(li, :) = ci1_med_all;
288     CI2_mean_all(li, :) = ci2_mean_all;
289     CI2_med_all(li, :) = ci2_med_all;
290
291     CI1_mean_tr(li, :) = ci1_mean_tr;
292     CI1_med_tr(li, :) = ci1_med_tr;
293     CI2_mean_tr(li, :) = ci2_mean_tr;
294     CI2_med_tr(li, :) = ci2_med_tr;
295
296     printf(' Full trace, type 1: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
297           ci1_mean_all(1), ci1_mean_all(2), ci1_med_all(1), ci1_med_all(2));
298     printf(' Full trace, type 2: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
299           ci2_mean_all(1), ci2_mean_all(2), ci2_med_all(1), ci2_med_all(2));
300
301     printf(' After transient removal, type 1: mean CI [%.6f, %.6f], median CI [%.6f,
302           %.6f]\n', ...
303           ci1_mean_tr(1), ci1_mean_tr(2), ci1_med_tr(1), ci1_med_tr(2));
304     printf(' After transient removal, type 2: mean CI [%.6f, %.6f], median CI [%.6f,
305           %.6f]\n', ...
306           ci2_mean_tr(1), ci2_mean_tr(2), ci2_med_tr(1), ci2_med_tr(2));
307 endfor
308
309 % =====
310 % A.4 - Little's Law verification for lambda = 100 requests/sec
311 % =====
312
313 printf('\n=====');
314 printf('A.4 - Little's Law verification for lambda = 100 req/sec\n');
315 printf('=====');
316
317 cutoff_A4 = warmup_fraction_A4 * r.t_end;

```

```

316
317 [N_sim_full, R_full, N_little_full, err_full] = little_stats(r, lambda_A1, 0);
318 [N_sim_tr, R_tr, N_little_tr, err_tr] = little_stats(r, lambda_A1, cutoff_A4);
319
320 printf('Without transient removal:\n');
321 printf(' R = %.6f ms\n', R_full);
322 printf(' N_sim, time-weighted total jobs = %.6f\n', N_sim_full);
323 printf(' lambda * R = (%.6f req/ms) * %.6f ms = %.6f\n', ...
324         lambda_A1 / 1000, R_full, N_little_full);
325 printf(' Percent error = %.6f %%\n', err_full);
326
327 printf('\nWith transient removal:\n');
328 printf(' Cutoff = %.6f ms = %.6f sec, %.0f%% of run time\n', ...
329         cutoff_A4, cutoff_A4 / 1000, 100 * warmup_fraction_A4);
330 printf(' R = %.6f ms\n', R_tr);
331 printf(' N_sim, time-weighted total jobs after cutoff = %.6f\n', N_sim_tr);
332 printf(' lambda * R = (%.6f req/ms) * %.6f ms = %.6f\n', ...
333         lambda_A1 / 1000, R_tr, N_little_tr);
334 printf(' Percent error = %.6f %%\n', err_tr);
335
336 % =====
337 % Final summary
338 % =====
339
340 printf('\n===== \n');
341 printf('FINAL SUMMARY - Lab 5, Part A\n');
342 printf('===== \n');
343
344 printf('\nA.1 lambda = %d req/sec, %d fully served requests\n', lambda_A1,
345         max_req_A1);
346 printf(' t_end = %.6f ms = %.6f sec\n', r.t_end, r.t_end / 1000);
347 printf(' arrivals = %d, fully served = %d\n', r.n_arrived, r.n_served);
348 printf(' R1 = %.6f ms, R2 = %.6f ms, R = %.6f ms\n', R1_A1, R2_A1, R_A1);
349 printf(' throughput: type-1 = %.6f jobs/sec, type-2 = %.6f jobs/sec\n', tp1_A1,
350         tp2_A1);
351 printf(' time-weighted mean jobs: N1 = %.6f, N2 = %.6f, N = %.6f\n', ...
352         N1_A1, N2_A1, N_A1);
353
354 printf('\nA.2 Stability bound\n');
355 printf(' lambda < %.6f req/sec\n', lambda_stability_bound);
356
357 for k = 1:length(lambdas_A2)
358     lam = lambdas_A2(k);
359     rho = lam * S_total / 1000;
360
361     if rho < 1
362         printf(' lambda = %3d: rho = %.6f -> stationary\n', lam, rho);
363     else
364         printf(' lambda = %3d: rho = %.6f -> non-stationary\n', lam, rho);
365     endif
366 endfor
367
368 printf('\nA.3 95%% confidence intervals, n = %d replications\n', n_runs_A3);

```

```

367 printf(' Transient removal rule: remove first %.0f%% of each run, guided by MQS(t)
plots.\n', ...
368     100 * warmup_fraction_A3);
369
370 for li = 1:length(lambdas_A3)
371     lam = lambdas_A3(li);
372
373     printf('\n lambda = %d req/sec\n', lam);
374
375     printf(' Full trace:\n');
376     printf(' type 1: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
377         CI1_mean_all(li, 1), CI1_mean_all(li, 2), ...
378         CI1_med_all(li, 1), CI1_med_all(li, 2));
379     printf(' type 2: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
380         CI2_mean_all(li, 1), CI2_mean_all(li, 2), ...
381         CI2_med_all(li, 1), CI2_med_all(li, 2));
382
383     printf(' After transient removal:\n');
384     printf(' type 1: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
385         CI1_mean_tr(li, 1), CI1_mean_tr(li, 2), ...
386         CI1_med_tr(li, 1), CI1_med_tr(li, 2));
387     printf(' type 2: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
388         CI2_mean_tr(li, 1), CI2_mean_tr(li, 2), ...
389         CI2_med_tr(li, 1), CI2_med_tr(li, 2));
390 endfor
391
392 printf('\nA.4 Little''s Law, lambda = %d req/sec\n', lambda_A1);
393 printf(' Without transient removal:\n');
394 printf(' R = %.6f ms, N_sim = %.6f, lambda*R = %.6f, error = %.6f %%\n', ...
395     R_full, N_sim_full, N_little_full, err_full);
396 printf(' With transient removal:\n');
397 printf(' R = %.6f ms, N_sim = %.6f, lambda*R = %.6f, error = %.6f %%\n', ...
398     R_tr, N_sim_tr, N_little_tr, err_tr);
399
400 printf('\nDone.');
```

```

401
402 endfunction
403
404 % =====
405 % Local functions
406 % =====
407
408 function res = sim_webserver(lambda, max_req)
409     % Run one DES simulation of the web server.
410     %
411     % lambda : external request arrival rate in requests/sec
412     % max_req : stop after this many requests have fully finished
413     %
414     % Returned fields:
415     % log_t, log_q1, log_q2, log_arrived, log_served
416     % resp_t1, resp_t2, total_resp
417     % completed_depart2, completed_arrival
418     % n_arrived, n_served, t_end
419

```



```

420     ARRIVAL = 1;
421     DEPART1 = 2;
422     DEPART2 = 3;
423
424     inter_arr_mean = 1000 / lambda;      % ms
425
426     % Event list rows: [time, event_type]
427     events = [sample_exp_mean(inter_arr_mean), ARRIVAL];
428
429     % FIFO queue rows: [job_type, phase_start_time, request_id]
430     queue = [];
431
432     busy = false;
433     cur_job = [0, 0, 0];                  % [job_type, phase_start_time, request_id]
434
435     % Logs start at t = 0 with an empty system.
436     log_t      = 0;
437     log_q1     = 0;
438     log_q2     = 0;
439     log_arrived = 0;
440     log_served  = 0;
441
442     resp_t1 = [];
443     resp_t2 = [];
444     total_resp = [];
445
446     arrival_times = [];
447     depart1_times = [];
448
449     completed_ids = [];
450     completed_arrival = [];
451     completed_depart2 = [];
452
453     n_arrived = 0;
454     n_served  = 0;
455     t = 0;
456
457     while n_served < max_req
458         if isempty(events)
459             break;
460         endif
461
462         % Pick the event with the smallest timestamp.
463         [t, idx] = min(events(:, 1));
464         etype = events(idx, 2);
465         events(idx, :) = [];
466
467         if etype == ARRIVAL
468             n_arrived = n_arrived + 1;
469             req_id = n_arrived;
470
471             arrival_times(req_id) = t;
472
473             % Schedule the next external arrival.

```

```

474         events = [events; t + sample_exp_mean(inter_arr_mean), ARRIVAL];
475
476         if ~busy
477             busy = true;
478             cur_job = [1, t, req_id];
479             events = [events; t + sample_lognormal(1, 1), DEPART1];
480         else
481             queue = [queue; 1, t, req_id];
482         endif
483
484     elseif etype == DEPART1
485         req_id = cur_job(3);
486
487         depart1_times(req_id) = t;
488         resp_t1 = [resp_t1, t - cur_job(2)];
489
490         % The same request becomes a type-2 job at the back of the FIFO queue.
491         queue = [queue; 2, t, req_id];
492
493         % Start the next job from the FIFO queue.
494         [cur_job, queue] = pop_next(queue);
495
496         if cur_job(1) == 1
497             events = [events; t + sample_lognormal(1, 1), DEPART1];
498         else
499             events = [events; t + sample_uniform(0.5, 1.0), DEPART2];
500         endif
501
502     elseif etype == DEPART2
503         req_id = cur_job(3);
504
505         resp_t2 = [resp_t2, t - cur_job(2)];
506         total_resp = [total_resp, t - arrival_times(req_id)];
507
508         completed_ids = [completed_ids, req_id];
509         completed_arrival = [completed_arrival, arrival_times(req_id)];
510         completed_depart2 = [completed_depart2, t];
511
512         n_served = n_served + 1;
513
514         if ~isempty(queue)
515             [cur_job, queue] = pop_next(queue);
516
517             if cur_job(1) == 1
518                 events = [events; t + sample_lognormal(1, 1), DEPART1];
519             else
520                 events = [events; t + sample_uniform(0.5, 1.0), DEPART2];
521             endif
522         else
523             busy = false;
524             cur_job = [0, 0, 0];
525         endif
526     endif
527

```

```

528         % Count type-1 and type-2 jobs in the system: FIFO queue + current service.
529         if isempty(queue)
530             nq1 = 0;
531             nq2 = 0;
532         else
533             nq1 = sum(queue(:, 1) == 1);
534             nq2 = sum(queue(:, 1) == 2);
535         endif
536
537         nq1 = nq1 + (cur_job(1) == 1);
538         nq2 = nq2 + (cur_job(1) == 2);
539
540         log_t      = [log_t,      t];
541         log_q1     = [log_q1,     nq1];
542         log_q2     = [log_q2,     nq2];
543         log_arrived = [log_arrived, n_arrived];
544         log_served  = [log_served,  n_served];
545     endwhile
546
547     res.log_t      = log_t;
548     res.log_q1     = log_q1;
549     res.log_q2     = log_q2;
550     res.log_arrived = log_arrived;
551     res.log_served  = log_served;
552
553     res.resp_t1 = resp_t1;
554     res.resp_t2 = resp_t2;
555     res.total_resp = total_resp;
556
557     res.completed_ids = completed_ids;
558     res.completed_arrival = completed_arrival;
559     res.completed_depart2 = completed_depart2;
560
561     res.n_arrived = n_arrived;
562     res.n_served  = n_served;
563     res.t_end = t;
564 endfunction
565
566 function [job, queue_out] = pop_next(queue)
567     % FIFO pop.
568     job = queue(1, :);
569     queue_out = queue(2:end, :);
570 endfunction
571
572 function x = sample_exp_mean(mean_value)
573     % Exponential random variable with the given mean.
574     u = rand();
575
576     while u <= 0
577         u = rand();
578     endwhile
579
580     x = -mean_value * log(u);
581 endfunction

```

```

582
583 function x = sample_lognormal(mu, sigma)
584     % LogNormal(mu, sigma), where mu/sigma are the parameters of the
585     % underlying normal distribution.
586     x = exp(mu + sigma * randn());
587 endfunction
588
589 function x = sample_uniform(a, b)
590     % Uniform(a,b).
591     x = a + (b - a) * rand();
592 endfunction
593
594 function avg = tw_mean(vals, times)
595     % Time-weighted average of a step function.
596     % vals(i) is assumed to hold over [times(i), times(i+1)).
597     avg = tw_mean_interval(vals, times, times(1), times(end));
598 endfunction
599
600 function avg = tw_mean_interval(vals, times, t_start, t_stop)
601     % Time-weighted average of a step function over [t_start, t_stop].
602     vals = vals(:)';
603     times = times(:)';
604
605     if length(times) < 2
606         avg = NaN;
607         return;
608     endif
609
610     t_start = max(t_start, times(1));
611     t_stop = min(t_stop, times(end));
612
613     if t_stop <= t_start
614         avg = NaN;
615         return;
616     endif
617
618     area = 0;
619     total_time = 0;
620
621     for i = 1:(length(times) - 1)
622         a = max(times(i), t_start);
623         b = min(times(i + 1), t_stop);
624
625         if b > a
626             area = area + vals(i) * (b - a);
627             total_time = total_time + (b - a);
628         endif
629     endfor
630
631     if total_time <= 0
632         avg = NaN;
633     else
634         avg = area / total_time;
635     endif

```

```

636 endfunction
637
638 function mqs = running_tw_mean(vals, times)
639     % Running time-weighted average:
640     %  $MQS(t_j) = (1 / (t_j - t_0)) * \int_{t_0}^{t_j} Q(s) ds.$ 
641     vals = vals(:)';
642     times = times(:)';
643
644     n = length(times);
645     mqs = zeros(1, n);
646
647     if n < 2
648         return;
649     endif
650
651     area = 0;
652     t0 = times(1);
653     mqs(1) = vals(1);
654
655     for i = 1:(n - 1)
656         dt = times(i + 1) - times(i);
657
658         if dt < 0
659             error('Times must be non-decreasing.');
660         endif
661
662         area = area + vals(i) * dt;
663         denom = times(i + 1) - t0;
664
665         if denom > 0
666             mqs(i + 1) = area / denom;
667         else
668             mqs(i + 1) = mqs(i);
669         endif
670     endfor
671 endfunction
672
673 function [ci_mean, ci_med] = calc_ci(xs, alpha)
674     % Confidence intervals for mean and median.
675     %
676     % Mean CI:
677     %  $\bar{x} \pm t_{\{n-1, 1-\alpha/2\}} * s/\sqrt{n}$ 
678     %
679     % Median CI:
680     % order-statistics interval  $[X_{(k)}, X_{(n+1-k)}]$  with
681     %  $k = \text{floor}(n/2 - z*\sqrt{n}/2).$ 
682
683     if nargin < 2
684         alpha = 0.05;
685     endif
686
687     xs = xs(:);
688     xs = xs(~isnan(xs));
689     xs = sort(xs);

```

```

690
691     n = length(xs);
692
693     if n < 2
694         ci_mean = [NaN, NaN];
695         ci_med  = [NaN, NaN];
696         return;
697     endif
698
699     xbar = mean(xs);
700     s = std(xs);
701
702     eta = t_quantile(1 - alpha / 2, n - 1);
703     ci_mean = [xbar - eta * s / sqrt(n), xbar + eta * s / sqrt(n)];
704
705     z = normal_quantile(1 - alpha / 2);
706     k = floor(n / 2 - z * sqrt(n) / 2);
707     k = max(k, 1);
708     k = min(k, floor((n + 1) / 2));
709
710     ci_med = [xs(k), xs(n + 1 - k)];
711 endfunction
712
713 function q = normal_quantile(p)
714     % Normal quantile. Uses norminv if available, otherwise erfinv formula.
715     if exist('norminv', 'file') || exist('norminv', 'builtin')
716         q = norminv(p);
717     else
718         q = sqrt(2) * erfinv(2 * p - 1);
719     endif
720 endfunction
721
722 function q = t_quantile(p, df)
723     % Student-t quantile.
724     % Uses tinv if available. If not available, uses standard normal as a
725     % fallback, except for df = 29 where the common n = 30 value is included.
726     if exist('tinv', 'file') || exist('tinv', 'builtin')
727         q = tinv(p, df);
728     else
729         if df == 29 && abs(p - 0.975) < 1e-12
730             q = 2.045230;
731         else
732             q = normal_quantile(p);
733         endif
734     endif
735 endfunction
736
737 function [N_sim, R_mean, N_little, err_percent] = little_stats(r, lambda, cutoff_ms)
738     % Little's Law verification for one trace.
739     %
740     % N_sim:
741     %   time-weighted mean total number of jobs in the system.
742     %
743     % R_mean:

```

```

744 % mean end-to-end response time of requests completed after cutoff.
745 %
746 % N_little:
747 % lambda_ms * R_mean.
748 %
749 % err_percent:
750 % relative absolute error between N_little and N_sim.
751
752 q_total = r.log_q1 + r.log_q2;
753
754 if cutoff_ms <= 0
755     N_sim = tw_mean(q_total, r.log_t);
756     idx = true(size(r.total_resp));
757 else
758     N_sim = tw_mean_interval(q_total, r.log_t, cutoff_ms, r.t_end);
759     idx = (r.completed_arrival >= cutoff_ms) & (r.completed_depart2 >= cutoff_ms);
760 endif
761
762 if sum(idx) == 0
763     R_mean = NaN;
764     N_little = NaN;
765     err_percent = NaN;
766     return;
767 endif
768
769 R_mean = mean(r.total_resp(idx));
770
771 lambda_ms = lambda / 1000;
772 N_little = lambda_ms * R_mean;
773
774 if abs(N_sim) < eps
775     err_percent = NaN;
776 else
777     err_percent = 100 * abs(N_little - N_sim) / abs(N_sim);
778 endif
779 endfunction
780

```